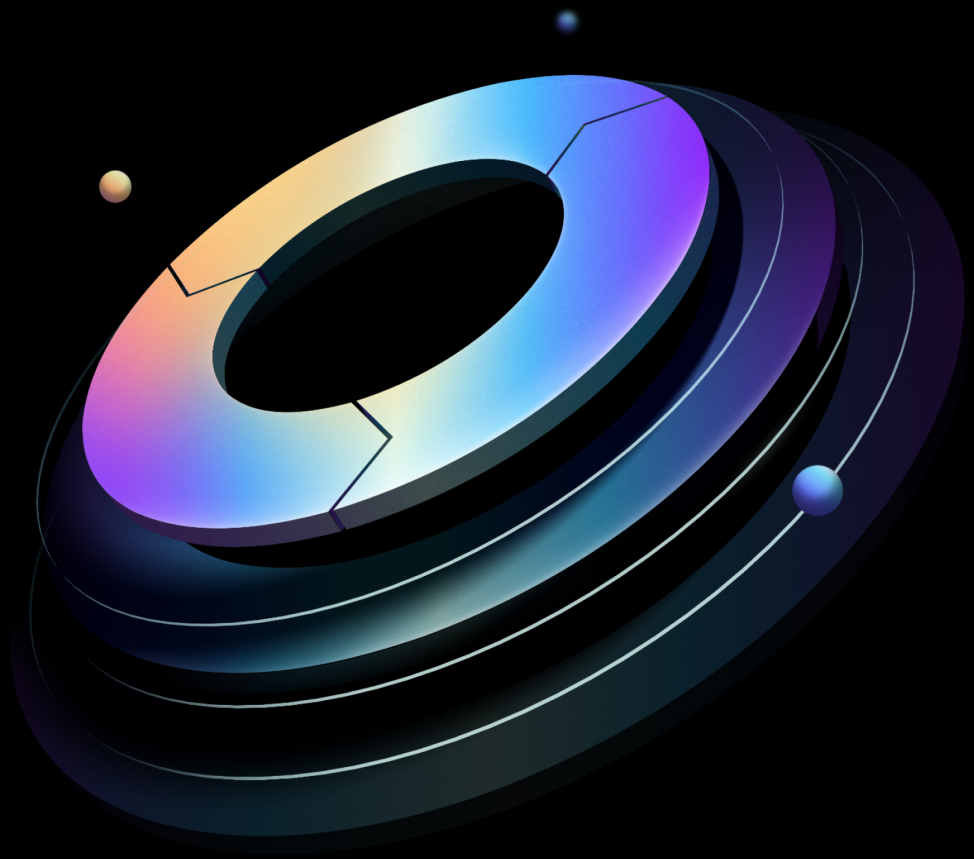
HashiCorp

Nomad Administration Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **4**
1.1 Why use HashiCorp Validated Designs? . . . 4
1.2 Prerequisites . . . 4
1.3 Objectives . . . 5
1.3.1 Business objectives . . . 5
1.3.2 Functional objectives . . . 5
1.4 Onboarding/adoption checklist . . . 5
1.4.1 Project checklist . . . 6

2 People and process **8**

3 Initial configuration **9**
3.1 Configure Nomad as code . . . 9
3.1.1 Terraform . . . 10
3.1.2 API and the command-line tool . . . 10
3.2 GitOps repository structure . . . 10

4 Identity and access management **12**
4.1 ACL management . . . 14
4.2 SSO using OIDC . . . 15
4.3 Workload identity . . . 16
4.3.1 Internal . . . 16
4.3.2 Third party OIDC-compatible systems . . . 16

5 Consumer onboarding **16**
5.1 Onboarding checklist . . . 17
5.2 Adoption checklist . . . 17

6 Networking configuration **17**

7 Scheduling **18**
7.1 Scheduling algorithms . . . 18
7.2 Node pool-level configuration . . . 19
7.3 Preemption configuration . . . 20
7.4 Memory oversubscription . . . 21

8 Nomad Enterprise observability 23
8.0.1 Metrics . . . 24
8.0.2 Logs . . . 24
8.0.3 Tracing . . . 25

9 Observability of the Nomad cluster 25
9.1 Metrics to monitor . . . 25
9.2 Operational logs and audit logs . . . 26
9.2.1 Operational logs . . . 26
9.2.2 Audit logs . . . 27
9.3 Observability of workloads running scheduled by Nomad Enterprise . . . 27
9.3.1 Pertinent metrics . . . 27
9.3.2 Logs . . . 28
9.3.3 Tracing . . . 30
9.4 Observability of the underlying infrastructure . . . 31
9.5 Who watches the watchman . . . 31
9.6 Benchmarking Nomad Enterprise . . . 32
9.7 Reference/useful links . . . 32

10 Backups 32

11 Overview 33
11.1 Prerequisites . . . 33

12 Example pipeline 34
12.1 Autopilot recommended configuration options . . . 37
12.1.1 Cleanup dead server . . . 37
12.1.2 Last contact threshold . . . 38
12.1.3 Server stabilization time . . . 38
12.1.4 Max trailing logs . . . 39
12.1.5 Server stabilization time . . . 39
12.1.6 Enable redundancy zones . . . 40
12.1.7 Disable upgrade migration . . . 40
12.1.8 Disable upgrade migration and enable custom upgrades . . . 40
12.2 Summary . . . 41
12.3 Resource quotas . . . 41

12.4 Quota specification . . . 41
12.5 Applying quotas to namespaces . . . 42
12.5.1 Version control . . . 42
12.6 Monitoring . . . 43
12.7 ACLs . . . 43
12.8 Federated clusters . . . 43
12.8.1 Communication . . . 44
12.9 Using Sentinel policy as code for Nomad . . . 44
12.10 When to Use Sentinel and when to use access control lists . . . 45
12.11 Best practices for authoring Sentinel policies . . . 45
12.11.1 The new Sentinel UI in Nomad . . . 46
12.12 Example Sentinel policies . . . 46
12.13 Control plane scaling . . . 46
12.14 Metrics to monitor . . . 46
12.15 Vertical scaling . . . 48
12.16 Horizontal scaling . . . 49
12.16.1 Prerequisites . . . 49
12.17 Enhanced read-only replica nodes . . . 50
12.17.1 Configure read-only replica nodes . . . 50
12.17.2 Autoscaler . . . 50
12.17.3 Prerequisites . . . 51
12.17.4 Example read only Autoscaling scaling policy for server nodes . . . 52
12.18 Federated clusters . . . 54
12.18.1 Scaling strategy summary . . . 55

13 Changelog **56**
13.1 2026-08 . . . 56
13.2 2026-01 . . . 56
13.3 2025-10 . . . 56
13.4 2025-07 . . . 56
13.5 2025-05 . . . 57
13.6 2025-04 . . . 57
13.7 2025-02 . . . 57

14 Credits **57**

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Prerequisites

Review HashiCorp's Cloud Operating Model which enables your organization to unlock the fastest path to value in a modern multi-cloud datacenter.

Note

This guide assumes that you have reviewed and implemented the following HVDs:* Nomad: Installation Guide (Self-Managed)

1.3 Objectives

You are implementing Nomad Enterprise to achieve your company's business and functional objectives. Below are the expected business benefits from implementing the recommendations detailed in this guide.

1.3.1 Business objectives

1. **Reduce time to market:** This guide assists you in establishing a robust standard workflow for deploying and managing the lifecycle of hybrid/multi cloud application deployments. When implemented effectively, developers can deploy applications more efficiently, reducing the time it takes for your organization to introduce new products and features to the market.
2. **Consistent compliance**: Through policy-as-code, organizations achieve compliant deployment flow, automate audit, and respond proactively to regulatory change.
3. **Improve skills and retention**: Through deployment templates reuse, organizations reduce the cognitive load associated with onboarding new talent and retain that talent longer by improving productivity for team members.
4. **Optimize cloud cost**: Achieve this by implementing a central shared service for application deployment. Standardize and enforce best practices by providing visibility into resources assigned to applications across the organization.

1.3.2 Functional objectives

1. Adopt a mature golden workflow for application deployment.
2. Enhance security posture.
3. Improve traceability of actions and ensure audit readiness.

1.4 Onboarding/adoption checklist

Follow the tasks below to accomplish a successful onboarding and adoption process for Nomad Enterprise. The time taken to complete these varies depending on the complexity of your organization and the level of executive alignment. However, we have found that using HashiCorp Professional Services or partner-provided services accelerates the process.

1.4.1 Project checklist

1. Identify key people from the Platform Team who own and operate Nomad. In your organization, the Platform Team may own the architecture but the Production Services/Support Team may own 24/7 day-to-day operations. Engage both teams at the outset.
2. Identify key executives sponsoring this project.
3. Establish a cadence call once a week/fortnight with the HashiCorp account team.
4. Conduct a quarterly business review with sponsoring executives.
5. We recommend that key platform team members attend HashiCorp Academy training. This training also enables the Platform Team as trainers for your organization.
6. Train application teams either by the Platform Team "trainers" or attend free hands-on workshops offered by HashiCorp solution engineers and architects.
7. Create a schedule for onboarding business units and/or application teams. We recommend for the *adopt* phase that you start with one or a handful of business units (see note below).
8. Establish key milestones to track progress. We recommend the following key milestones:
    - Nomad onboarding.
    - Platform team enablement.
    - Application team early adopter enablement.
    - Application team early adopter onboarding.

Tip

Onboarding business units/application teams HashiCorp recommends that you base the initial business unit/application team onboarding schedule on a representative set of early adopter teams with one or more of the following characteristics:* A high incentive to use Nomad application scheduling to achieve their goals. * Have a DevOps skill set.Those characteristics increase the chances of success.The schedule introduces around five teams in the first set, working with them from development to UAT. Introduce a second set of 10-20 teams into development. This second set would benefit from the feedback the more experienced first set provides as part of pipeline refinement.When the first set has reached the production environment and the second set has reached UAT, introduce a third set of around 50-100 application teams into development. These benefit from the further refinement brought through feedback and collaboration with the first and second sets of teams. Through working with this third set, increased scale is both visible and demonstrable to senior management, standardizing, adding efficiency gains and cost savings and making project success highly likely.

Onboarding checklist

1. Configure core Nomad Enterprise components.
    - Servers
    - Clients
    - Access control lists
    - Logs and metrics collection
2. Establish a workflow to onboard application teams to Nomad Enterprise.
    - Deployment templates and best practices
    - Observability patterns
3. Conduct initial discussions with the first set of early adopters regarding their experience. Update the project backlog with next step improvements.

Adoption checklist

1. Determine the migration roadmap for existing applications.

2. Establish a workflow for application teams.
3. Complete a production readiness assessment with a HashiCorp solution architect.

Tip

We highly recommend arranging a meeting with a HashiCorp solution architect soon after acquiring your license. They offer essential architectural and integration guidance for your project.Schedule another session within three months to ensure that the transition architecture and migration plans remain clear and unimpeded.Regular meetings with your HashiCorp solution engineer throughout the project and into production are also advised to continually enhance business value.

2 People and process

Identify key people from the Platform Team who own and operate Nomad. In your organization, the Platform Team may own the architecture but the Production Services/Support Team may own 24/7 day-to-day operations. Engage both teams at the outset.

1. Identify key executives sponsoring this project.
2. Establish a cadence call once a week/fortnight with the HashiCorp account team.
3. Conduct a quarterly business review with sponsoring executives.
4. We recommend that key platform team members attend HashiCorp Academy training. This training also enables the Platform Team as trainers for your organization.
5. Train application teams either by the Platform Team "trainers" or attend free hands-on workshops offered by HashiCorp solution engineers and architects.
6. Create a schedule for onboarding business units and/or application teams.
7. Establish key milestones to track progress.

3 Initial configuration

> **Note**
>
> This page covers ongoing operator governance and configuration-as-code workflows for Nomad Enterprise. For first-run bootstrap tasks after deployment, see the Installation Guide: Post-installation configuration.

This section describes how to configure the product after installation. It may take several weeks to work through all of the recommendations from each section, but this is the primary focus of the Nomad Enterprise platform operators after completing the initial deployment of Nomad Enterprise. Establishing these foundational configuration best practices before targeting consumer workflows is crucial. This ensures a stable foundation for both current and future growth.

After deploying and bootstrapping Nomad Enterprise, follow these steps to configure the servers.

1. Enable mTLS and gossip encryption
2. Enable and configure access control lists and OIDC for RBAC
3. Set up automated upgrades to ensure Nomad cluster stays up-to-date with the latest features and security patches
4. Implement a robust backup strategy to protect your Nomad cluster's state and configuration
5. Load balance and reverse proxy for Nomad's web UI
6. Use Terraform to manage Nomad configuration as code, enabling version control and reproducibility

3.1 Configure Nomad as code

When managing Nomad Enterprise configurations, prioritize automation from the start. While organizations may adopt different GitOps and configuration management practices, avoiding manual changes to your production Nomad Enterprise environment to minimize inconsistencies and control change promotion across environments.

Implement a Git-based workflow as the primary method for modifying Nomad configuration through pull or merge requests.

1. Block direct commits to the main branch linked to production configuration.
2. Require approvals and code reviews for pull requests.

3. Restrict manual operator-level changes to reduce drift between code-defined configuration and real-world state.
4. Stick to a single configuration style.

There are two main approaches for configuring Nomad Enterprise:

3.1.1 Terraform

- Manages configuration state, maintaining a system of record and single source of truth, overwriting any changes made when using a different configuration method.
- HCP Terraform and Terraform Enterprise customers can leverage advanced self-service capabilities such as private module registry, no-code modules, and Sentinel.
- Preferred for automating day 2 resources, such as namespaces, node pools, and so on.
- Provides a single central place to manage configurations.

3.1.2 API and the command-line tool

- Covers all configuration options.
- Requires additional logic for dependencies, creation ordering, and existing state management.
- More challenging and time consuming to build configuration workflows.

Choose Terraform as the primary method, as it simplifies management and provides better control. However, the API and the command-line tool are valuable for configurations not covered by Terraform or for specific use cases requiring fine-grained control.

3.2 GitOps repository structure

In the earlier stages of adoption, it is common for the Platform Team operating Nomad to store policies, roles and configuration values in one monolithic configuration repository. An example of this is below.

```
nomad-config/├──
 main.tf├──
 variables.tf├──
 outputs.tf├──
 node_pools/│├──
  general.tf│└──
  gpu.tf├──
 namespaces/│├──
  production.tf│├──
  staging.tf│└──
  development.tf├──
 resource_quotas/│├──
  production_quota.tf│├──
  staging_quota.tf│└──
  development_quota.tf└──
 sentinel_policies/├──
   enforce_docker.tf└──
   restrict_job_priority.tf
```

As adoption matures, consider splitting your repository based on specific components or teams. Organizations anticipating significant Nomad adoption must implement a distributed repository structure from the beginning. This approach involves creating separate repositories for different components or teams, rather than starting with a monolithic configuration repository. This strategy provides better scalability and delegation of responsibilities as your Nomad usage expands.

```
nomad-core-config/├──
 main.tf├──
 variables.tf├──
 outputs.tf├─
nomad-sentinel-policies/
| ├── main.tf
| ├── enforce_docker.tf
| └── restrict_job_priority.tf└─
nomad-node-pools/├──
   general.tf└──
   gpu.tf

nomad-team-namespaces/├──
 main.tf├──
 team_alpha.tf├──
 team_beta.tf└──
 team_gamma.tf
nomad-team-resource-quotas/├──
 main.tf├──
 team_alpha_quota.tf├──
 team_beta_quota.tf└──
 team_gamma_quota.tf
```

These repository structures are basic examples to get started. Refer to the Terraform Style Guide for Terraform implementation best practices.

For more detailed information on Terraform modules and workflows, visit the Terraform Validated Design pages at Infrastructure-as-code and cloud provisioning and Terraform Workflows.

4 Identity and access management

The access control lists (ACL) system in Nomad provides authentication and authorization. This gives Nomad administrators fine-grained control over what users and applications can do within the Nomad environment.

> **Note**
>
> ACLs are not enabled by default, but are a crucial configuration setting for any production environment, and an important pillar of the Nomad security model

Properly structuring ACL policies and roles ensures that only authorized users and applications can

interact with the Nomad cluster, and only to the level they need to. This helps prevent accidental or malicious actions that could disrupt the cluster or compromise sensitive data.

Structuring ACLs is organization-dependent. These are the best practices.

- Create role-based policies:
  - Define roles like *developer*, *operator*, and *admin* instead of one-off tokens.
- Use the principle of least privilege:
  - Grant minimal permissions necessary for each role.
- Use namespaces:
  - Use namespaces to isolate different tenants, teams, applications or environments.
- Use wildcards judiciously:
  - Be cautious with wildcards to avoid overly broad permissions.
- Avoid static/long-lived tokens:
  - Use either short lived Nomad ACL tokens, the Vault Nomad secrets engine for time-bound dynamic tokens, or Nomad single sign-on with OIDC which allows trusting external OIDC-capable identity providers, such as Okta, Auth0, EntraID for human access, or GitHub Actions, GitLab CI or Vault for machine access.
- Implement a robust offboarding process for any static tokens:
  - Ensure timely removal of access when users leave/applications no longer need access.
- Regular access audits:
  - Review ACL policies and any static tokens periodically.
- Enable audit logs:
  - Use audit logs to monitor and detect suspicious activity.
- Use Sentinel:
  - Sentinel is a policy as code framework that allows you to enforce fine-grained policies at job deploy time. Use these to enforce extra security or business logic (such as to only authorize Docker containers from a specific Docker registry, or disallow deploys on Friday afternoon, and so on.)

- Treat namespaces and ACL policies as code.
    - Store them in a version control system, ideally manage them with Terraform, and use CI/CD pipelines to deploy them to Nomad. Review and audit changes and apply them consistently across all environments.

4.1 ACL management

ACLs comprise of capabilities, policies, roles, and tokens.

Attach policies to tokens through the correct definition of roles. Use tokens to authenticate to Nomad (use Vault or an SSO identity provider to broker this authentication). Policies control which actions users can access in Nomad Enterprise.

Use Terraform to manage namespaces, ACL policies and roles, as it allows for easier version control and code review, as well as for an easier understanding of all the policies in use in the Nomad Enterprise cluster. Used Terraform to also enable a self-service model using Terraform child modules, and to render common combinations of namespace/associated policies.

An example of a policy that allows submitting jobs and a few related activities in a specific namespace, using raw HCL for Nomad or Terraform is below.

```
# Allow reading resources within the "dev" namespace as well as submitting
# jobs to this namespace, without allowing access to view log output or inspect
# the filesystem.
namespace "dev" {
  capabilities = ["submit-job", "list-jobs", "read-job", "parse-job"]
}
```

Create such a policy in Nomad using the command-line tool or API as follows.

```
nomad acl policy apply -description "Allow reading resources within the 'dev'
    namespace as well as submitting jobs to this namespace, without allowing access
    to view log output or inspect the filesystem." dev-policy dev.hcl
```

Create the same policy using the Terraform `nomad_acl_policy` resource from the Nomad Terraform provider.

```
resource "nomad_acl_policy" "dev" {
  name        = "dev-policy"
  description = "Allow reading resources within the 'dev' namespace as well as
      submitting jobs to this namespace, without allowing access to view log output
      or inspect the filesystem."

  rules_hcl = <<EOT
namespace "dev" {
  capabilities = [
    "submit-job",
    "list-jobs",
    "read-job",
    "parse-job"
  ]
}
EOT
}
```

Attach the created policy to an ACL role, or use it directly by creating a token with it. Preferably, use SSO to authenticate users and applications to roles.

4.2 SSO using OIDC

Using an external identity provider (IdP) for authentication and authorization is a recommended pattern for security and simplicity. Nomad supports OIDC (OpenID Connect) for Single Sign-On (SSO) with external identity providers such as Okta, Auth0, EntraID, and so on. for human auth, and GitHub Actions, GitLab CI for machine auth. This allows users, pipelines, and applications to authenticate to Nomad using their existing identity, and Nomad to trust the IdP to provide the necessary information about the user, such as their username, groups, metadata, and so on. This is more secure than static tokens, as it allows for centralized management of users and their access, which is short-lived and time bound. It also allows for revocation of access when users leave the organization or no longer need access.

To enable OIDC, you need to configure the necessary ACL policies and ACL roles in Nomad, and create an auth method (using the command-line tool or preferably, the `nomad_acl_auth_method` Terraform resource).

This is an example tutorial using Auth0. The same general principles apply to any OIDC provider.

4.3 Workload identity

The ACL system is what enables Nomad to provide Workload Identity to each workload running in Nomad, and serve as an OIDC identity provider. This allows to uniquely and securely authenticate and authorize workloads running in Nomad without any static credentials needed. This enables workloads running in Nomad to authenticate to Nomad itself, Vault, assume a role in AWS IAM/GCP IAM, or any other OIDC/JWT compatible system.

4.3.1 Internal

By default, each allocation in Nomad gets an identity which gives it implicit access to the Nomad API. This identity is unique to the allocation, and gives it read-only access to Nomad variables in the respective path for that allocation's job/group/task (`nomad/jobs/job/task_group/task`). Attach ACL policies explicitly to jobs/groups/tasks to enhance that identity, and provide granular access to alternative Nomad variable paths, or the Nomad Task API which allows for secure and local access to the Nomad API from within tasks (perfect for load balancers, control tasks, automation, and so on.)

4.3.2 Third party OIDC-compatible systems

Configure the OIDC issuer in Nomad to use workload identity for auth to OIDC-compatible third parties (such as Vault, AWS IAM, and GCP IAM). Configure trust between the external system and Nomad as an identity provider (IdP).

The exact flow and configuration depends on the third party system trusting Nomad.

- Vault: Nomad workloads can authenticate to the OIDC/JWT Vault Authentication method.
- AWS IAM: Nomad workloads can assume AWS IAM roles.
- GCP Workload Identity Federation: Federate GCP IAM identities to Nomad workloads.

5 Consumer onboarding

Follow the tasks below to accomplish a successful onboarding and adoption process for Nomad Enterprise. The time taken to complete these varies depending on the complexity of your organization

and the level of executive alignment. However, we have found that using HashiCorp Professional Services or partner-provided services accelerates the process.

5.1 Onboarding checklist

1. Configure core Nomad Enterprise components.
    - Servers
    - Clients
    - Access control lists
    - Logs and metrics collection
2. Establish a workflow to onboard application teams to Nomad Enterprise.
    - Deployment templates and best practices
    - Observability patterns
3. Conduct initial discussions with the first set of early adopters regarding their experience. Update the project backlog with next step improvements.

5.2 Adoption checklist

1. Determine the migration roadmap for existing applications.
2. Establish a workflow for application teams.
3. Complete a production readiness assessment with a HashiCorp solution architect.

6 Networking configuration

> **Note**
>
> This page covers the operator setup and management of Nomad Enterprise networking. For how workload authors consume networking capabilities in jobs, see the User Guide: Networking.

Nomad Enterprise operators should make a choice of networking mode based on infrastructure requirements and organizational policies. Regardless of the chosen networking mode, it is crucial

to ensure that all necessary networking configurations are present and available **before** deploying jobs. If there is a mistake in the networking configuration, when attempting to deploy jobs, errors are likely.

- For self-service models, it is ideal for application owners to have templated job definitions by Nomad Enterprise operators with the correct networking mode and port mappings.
- Coordinate with your network engineers regarding any necessary security groups or firewall configurations to ensure proper network access for your workloads.
- Familiarize yourself with the network plugins available in your Nomad Enterprise cluster. Different plugins may offer features like network policy enforcement or advanced routing capabilities.
- The Container Network Interface (CNI) plugin allows for more advanced networking configurations and is useful in multi-node clusters. It enables the use of various network plugins for different networking requirements.

7 Scheduling

> **Note**
>
> This page covers operator setup and management of Nomad scheduler behavior. For how workload authors influence placement in jobs, see the User Guide: Job placement.

7.1 Scheduling algorithms

Nomad Enterprise supports two primary scheduling algorithms.

1. Bin packing - Aims to maximize resource usage by packing workloads logically into available clients. It is ideal for cloud environments where infrastructure billing is time/resource based and can be scaled in and out.
2. Spread - This algorithm distributes jobs evenly across all available clients to reduce density and potential resource contention. It is suitable for environments where clients are pre-provisioned and scale gradually, such as on-premises deployments.

Configure these as a cluster-level default or at the node pool level.

It is important to distinguish between the spread stanza for task groups and the spread algorithm at the cluster level. The task group's spread option is for customizing the spreading allocations, which is, by default, client-based. The spread algorithm is for spreading out the jobs. The default behavior is to bin pack jobs together, and spread allocations of jobs.

Configuring scheduling algorithms

To set the scheduling algorithm at the cluster level, use either configure file, command-line tool, or API. If the cluster is not in a bootstrapped state, you can enable it by adding a `default_scheduler_config` section to your agent configuration file.

```
server {
  default_scheduler_config {
    scheduler_algorithm = "spread"
  }
}
```

7.2 Node pool-level configuration

Node pools in Nomad Enterprise allow you to customize the scheduler algorithm per node pool. This is useful for mixed environments where different node types require different scheduling strategies.

Configuring a node pool

1. Set your Nomad endpoint environment variable to the correct server address and login if needed.

   ```
   $ echo $NOMAD_ADDR
   https://<correct IP or hostname>:4646
   nomad login
   ```

2. Create a configuration file: `nodepools.nomad.hcl`.

```
node_pool {
  name = "cloud-pool"
  scheduler_config {
   scheduler_algorithm = "binpack"
  }
}

node_pool {
  name = "on-prem-pool"
  scheduler_config {
   scheduler_algorithm = "spread"
  }
}
```

3. Apply the node pool configuration to the cluster: `nomad node pool apply nomadpools.nomad.hcl`.
4. Add the `node_pool` parameter to the client configuration file to add the client to the node pool.

```
# client.hcl
client {
   node_pool = "cloud-pool" #change name to suit your naming convention
}
```

5. Restart the Nomad agent, that is, `systemctl restart nomad`.
6. Job's are now able to opt-in to a node pool by specifying the `node_pool` parameter

For production, version control your node pool configurations and apply them to your pipeline build process.

> **Note**
>
> Node pool configurations override the default scheduler configuration. For example, if setting the default cluster configuration to `binpack` and there is a node pool with `spread` set, then any workloads placed on the node pool use `spread`.

7.3 Preemption configuration

Preemption allows Nomad Enterprise to evict lower-priority tasks to make room for higher-priority tasks when resources are scarce. It ensures that critical workloads can acquire the necessary re-

sources even when the cluster is under high utilization. Nomad Enterprise enables this feature by default for `system` jobs.

We recommend enabling preemption on production clusters for all workload types if there are critical tier 1 workloads that require priority sharing hosts with lower tier workloads. This ensures that tier 1 workloads always receive priority over the expense of potentially having downtime for the lower tier workloads.

If the cluster is not in a bootstrapped state yet, you can enable preemption by adding a `default_scheduler_config` section to your agent configuration file.

```
server {
  default_scheduler_config {
    preemption_config {
      batch_scheduler_enabled    = true
      system_scheduler_enabled   = true
      service_scheduler_enabled  = true
      sysbatch_scheduler_enabled = true     }
  }
}
```

Command-line tool

- `nomad operator scheduler set-config -preempt-batch-scheduler=true`
- `nomad operator scheduler set-config -preempt-service-scheduler=true`
- `nomad operator scheduler set-config -preempt-sysbatch-scheduler=true`

API

`/v1/operator/scheduler/configuration` For additional details visit the Preemption documentation page.

7.4 Memory oversubscription

Memory oversubscription is an opt-in feature which allows tasks to exceed their reserved memory limit if the client has excess memory capacity. We recommend enabling this feature to help maximize cluster memory utilization while also allowing a margin of error in case a task has a sudden memory spike.

> **Tip**
>
> Enable this feature globally or per node pool

ExecV2, raw_exec, Docker, Podman, and Java task drivers support memory oversubscription. Consult the documentation of community-supported task drivers for their memory oversubscription support.

Visit the Oversubscribe Memory tutorial for more information on how to configure.

If the cluster is not in a bootstrapped state yet, you can enable the memory oversubscription by adding a `default_scheduler_config` section to your agent configuration file.

```
server {
  default_scheduler_config {
    memory_oversubscription_enabled = true
  }
}
```

Command-line tool: [nomad operator scheduler set-config -memory-oversubscription =true](https://developer.hashicorp.com/nomad/commands/operator/scheduler/set-config#memory-oversubscription)

API: [/v1/operator/scheduler/configuration](https://developer.hashicorp.com/nomad/api-docs/operator/scheduler#update-scheduler-configuration)

Node pool-level configuration

```
node_pool {
  name = "cloud-pool"
  scheduler_config {
   memory_oversubscription_enabled = true
  }
}

node_pool {
  name = "on-prem-pool"
  scheduler_config {
   memory_oversubscription_enabled = false
  }
}
```

Task configuration

Tasks must specify `memory_max` to take advantage of memory oversubscription.

```
job "example-job" {
  group "example" {
    task "server" {
      resources {
        cpu                  = 100
        memory           = 256
        memory_max = 768

      }
    }
  }
}
```

Additional recommendations

To avoid degrading the cluster experience, we recommend examining and monitoring resource utilization and considering the following suggestions:

- Set `oom_score_adj` for Linux host services that are not managed by Nomad Enterprise, for example Docker, logging services, and the Nomad agent itself. For `systemd` services, you can use the `OOMScoreAdj` field.
- Monitor hosts for memory utilization and set alerts on out-of-memory errors
- Set the client reserved with enough memory for host services that are not managed by Nomad as well as a buffer for the memory excess. For example, if the client reserved memory is 1GB, the allocations on the host may exceed their soft memory limit by almost 1GB in aggregate before the memory becomes contended and allocations get killed.
- Leverage resource quotas to restrict resource utilization within a namespace.

Remember to thoroughly test and validate these configurations in a non-production environment before applying them to your production Nomad cluster. Monitor the cluster's performance and resource utilization and make adjustments based on your specific workload requirements.

8 Nomad Enterprise observability

Observability is a crucial aspect of operating any system. In the context of Nomad Enterprise, it allows operators to gain insights into the health, performance, and behavior of both the Nomad

Enterprise cluster itself and the workloads it manages.

> **Tip**
>
> The three pillars of observability are:1) Metrics: Quantitative measurements of system behavior over time. 2) Logs: Detailed records of events and actions within the system. 3) Tracing/sometimes overlapping with Application Performance Monitoring (APM): The ability to follow a request or transaction through a distributed system.

In a Nomad Enterprise environment, these pillars apply as follows:

8.0.1 Metrics

- Nomad-provided metrics help monitor cluster health, resource utilization, and job performance.
- Application metrics are typically handled by applications and their developers, and help monitor application-specific behavior and metrics, which can be technical (number of requests per second, latency) or business (number of orders processed, revenue, and so on.)
- System metrics provided by the underlying infrastructure (be it bare metal or virtualized) help monitor the health and performance of the infrastructure on which Nomad Enterprise runs.

Having all three in a single system and being able to dashboard and alert on them together is key to having a complete understanding. As an example, the underlying infrastructure's I/O performance impacts the application request latency. Seeing a spike in the latter can be hard to explain without knowledge of the former. Also use metrics to ensure that the cluster can handle the load its under and is thus useful for Autoscaling and to ensure there is minimal waste.

8.0.2 Logs

- Nomad-provided logs come in two types - operational logs, which provide detailed information about Nomad operations (evaluations, scheduling allocations, provisioning storage/networking, operating on templates, and so on) and audit logs which are useful to keep track of human/machine actions in Nomad.
- The applications themselves generate application logs and help understand application-specific behavior and events.

- The underlying operating system provides logs and related infrastructure if it exists (for example, a hypervisor if running in VMs), and these help us understand the health of the infrastructure on which Nomad runs.
- Having all three in a single system and being able to search and correlate them together is key to having a complete understanding. As an example, a network card driver issue (logged about in the OS/hypervisor logs) can cause network-related errors in both Nomad and application logs, but they wouldn't be aware of it, and neither would an operator with access only to them.

8.0.3 Tracing

Tracing allows you to understand the flow of requests through applications deployed on Nomad. This is useful in a microservices architecture where a single request can span multiple services. By tracing the request, you can identify bottlenecks, latency issues, and other performance problems, and correlate them to metrics/logs explaining them.

Implementing a comprehensive observability strategy in Nomad Enterprise is crucial for both operators and application developers - it enables proactive issue detection, faster troubleshooting, and informed capacity planning, but also helps to better understand applications' behavior, identify bottlenecks and catch regressions.

The exact composition and configuration of observability tools and practices depend on the specific requirements of each Nomad Enterprise environment, the organization's needs and the existing tooling already in use. Most observability agents support multiple inputs and outputs. The following sections provide generally applicable good practices to follow. The key is to have a holistic view of the system, and to be able to correlate information and events across the three pillars to understand everything.

9 Observability of the Nomad cluster

9.1 Metrics to monitor

When monitoring the Nomad cluster itself, focus on the most crucial metrics as described in the Monitoring Nomad documentation. They include vital information about the health of the cluster

itself, as well as scheduling performance and bottlenecks. It is important to, alongside metric collection, set up alerting on the most important and relevant metrics. There is also a complete Metrics Reference available - some metrics might not be crucial, but can still be useful for debugging or correlations.

To enable the collection of these metrics, Nomad Enterprise provides the `telemetry` block in agent configurations. Metrics are available in a variety of formats, and either pushed to a compatible system/agent, or made available for pulling. Statsd and DataDog are the supported push formats. Prometheus format available for pulling metrics. Most modern metrics/observability agents support multiple metric formats, if nothing else, Prometheus (this applies for the OpenTelemetry Collector, Datadog Agent, Elastic Agent, Grafana Alloy, InfluxData Telegraf, New Relic, AppDynamics, Dynatrace and so on).

For example, to enable Prometheus-compatible metrics collection, you can add the following to your Nomad configuration

```
telemetry {
    publish_allocation_metrics = true # if metrics from the allocations running in
        Nomad should be published
    publish_node_metrics = true       # if metrics from Nomad nodes should be
        published
    prometheus_metrics = true         # if metrics should be made available in the
        Prometheus format on the /metrics path
}
```

9.2 Operational logs and audit logs

Operational logs provide detailed information about Nomad Enterprise's internal operations, while audit logs capture security-relevant events for compliance and forensics. Ship both to a centralised location to index them for search, ensure they survive node failures, and for correlation with other logs and metrics. We recommend your organisation's strategic log aggregation system.

9.2.1 Operational logs

- Server logs: Contain information about leadership changes, job submissions, and evaluations.
- Client logs: Include details about task starts, stops, related activities such as downloading container images, provisioning networking, attaching volumes, and so on.

To configure logging, adjust the `log_level` and `log_file` settings in your Nomad Enterprise agent configuration.

```
log_level = "INFO"
log_file = "/var/log/nomad/nomad.log"
```

Examine the settings used for log rotation and adjust accordingly, depending on the specifics of the environment such as available disk space.

9.2.2 Audit logs

- Capture all API requests and responses.
- Record authentication and authorization decisions.

Enable audit logging by adding an `audit` block to your Nomad Enterprise server configuration.

```
audit {
    enabled = true
    sink "file" {
        type = "file"
        format = "json"
        path = "/var/log/nomad/audit.log"
    }
}
```

9.3 Observability of workloads running scheduled by Nomad Enterprise

9.3.1 Pertinent metrics

Nomad Enterprise collects metrics about its workloads such as how much resource it is using. This is useful for a broad understanding of how the workloads are behaving, but it is not enough to understand the workload itself. For that, collect emitted application-specific metrics, but that is the responsibility of the application developers to expose.

For workloads running on Nomad Enterprise, focus on these generic metrics and anything else that is relevant to your applications.

1. Application performance (entirely up to application developers to expose):

- Request rates and latencies

- Error rates
- Custom application-specific metrics such as number of transactions, unique users, third-party API failure rates, and so on.

1. Resource consumption

- CPU and memory usage per task
- Number of allocations running/failed
- Number of allocations that were OOM (out of memory) killed
- Amount of time the allocation was CPU throttled

Nomad Enterprise allows you to expose the second set of metrics through its telemetry system, and the first set by integrating metric collection tools with Nomad Enterprise.

By example, a Prometheus Agent running as a `system` job on all Nomad Enterprise clients collects both the Nomad Enterprise-provided metrics from the local Nomad agent, and the application-provided ones. It either scrapes the application's metrics endpoint (discoverable through service discovery or agent-native application discovery) or by having the application push them to the agent over a local port/socket. This allows for useful scaling and consumption, with the agent being centrally managed and thus to update/reconfigure, and the applications only needing basic configuration.

9.3.2 Logs

By default, Nomad collects `stdout` and `stderr` from running allocations (storing them in the `alloc/logs` folder) and makes them available using the API, UI and command-line tool. This is useful for live debugging and troubleshooting, but it is not enough for a production system because the logs are not persisted (however, they are garbage collected). Such logs and are insufficiently searchable, and do correlate well with other allocations.

To have complete log management, ship these logs to a centralized log aggregation system like ElasticSearch/OpenSearch, Splunk or equivalent. As with everything else, ideally, this would be where Nomad Enterprise server and client logs, and OS logs are also shipped to, so you can correlate them with together.

There are a number of different ways to collect allocation logs from Nomad Enterprise, and the most appropriate one depends on the log aggregation system in use, the agent intermediaries, and

most importantly, the allocation *type* (for instance, Docker, Java, `exec`). Some methods work in all scenarios, others would be task driver specific.

Sidecar

Since each allocation has access to all of its logs in the `alloc/logs` folder, a sidecar task can be ran in the same task group as the main workload, that collects the logs and ships them to the log aggregation system. Do this with a sidecar container that runs a log shipper like Filebeat, Fluentd, Logstash, Vector, Promtail, or equivalent. It allows custom logic such as sampling (only send 50% of the logs), filtering (such as only error-level logs) and transformations (such as to remove specific labels or tags, add custom metadata such as node name or version or similar.) Also, since the sidecar shares the same environment, log enrichment is optional as it has access to all the same metadata (task/task group name, environment variables, cloud metadata, and so on). The main downside of this approach is that it is wasteful on resources (each allocation has its own logging agent), and that the configuration is *per-task group*, making it hard to scale without an advanced deployment flow implementing Nomad Pack and Pack dependencies or equivalent.

Task driver logging

Some task drivers, most notably Docker (see a list of available log destinations out of the box in the Docker documentation), support various logging options and logging plugins. Configure these to directly send logs to a supported backend such as AWS CloudWatch Logs, Splunk, syslog, or equivalent.

This is one of the most efficient ways to collect logs, as it does not require an intermediary agent. However it has a number of downsides. It relies on the task driver implementing the feature and thus does not work for heterogeneous workloads. The task driver ships directly as they are, therefore client-side sampling or transformations are out of scope. The configuration is also *per-task*, making it hard to scale without an advanced deployment flow implementing Nomad Pack and Pack dependencies or equivalent.

Agent integrating with task driver

Most logging agents come with direct support for some of the most popular task drivers (such as Docker) directly. This allows the agent to collect logs from the Docker Daemon directly (as orchestrated by Nomad Enterprise) and ship them to the log aggregation system. This is a good middle

ground between the two previous approaches - it is *per Nomad client* (the agent can run as a `system` job) and thus more efficient, does not require a sidecar, allows for custom transformations/filters/sampling, and can scale.

The downside is that it requires the agent to be able to integrate with the task driver, and thus it might not work for all task drivers. For Docker in particular, agents rely on Docker labels to enrich logs with metadata (to know which container the logs are from), and by default those only include the allocation id `extra_labels`. Configure them specifically to expose more metadata.

Agent integrating with Nomad Enterprise

The most advanced approach is to have the agent (running as a `system` job) integrate with Nomad Enterprise directly, and collect logs from the Nomad Enterprise API. This allows for the most flexibility, as the agent can collect logs from any allocation, regardless of the task driver, and do any transformations/filters required.

Configure it to collect logs from all allocations, or only those that match certain criteria. The downside here is that it requires the agent to be able to integrate with the Nomad Enterprise API, which limits the usable agents. At this time of writing, the only agent with direct support of Nomad Enterprise is Filebeat; for other agents, there are community utilities to bridge the gap (such as vector-logger for Vector, and nomad_follower for others).

Agent collecting logs from the filesystem

Nomad Enterprise clients store allocation folders locally in the `data_dir`. Any logging agent can also collect the logs directly from host the filesystem. This approach is not the best because the only metadata available is the allocation id, which is not enough information, and enriching with more relevant information such as the task/task group name is not straight forward to achieve.

9.3.3 Tracing

Implementing distributed tracing for microservices running on Nomad Enterprise provides end-to-end visibility into request flows, service dependencies, and performance bottlenecks. Tracing is crucial for distributed environments to have a clear picture of how everything works. Nomad Enterprise itself does not provide tracing, but it allows you to run popular tracing systems alongside your workloads.

Similar to metrics and log collection, an agent can be ran as a `system` job, and applications can send to it. A common pattern is to have the agent listen on a local (to the host) port/socket, and have all applications (on that host) sending traces to it. The agent collects them, performs any necessary sampling, transformations, and filters, and ships them to the tracing backend. This allows for scaling and consumption, with the agent being centrally managed and thus straight forward to update/reconfigure, and the applications only needing basic configuration.

9.4 Observability of the underlying infrastructure

Monitoring the underlying infrastructure is crucial for maintaining a healthy Nomad cluster:

1. Host-level metrics
    - CPU, memory and disk usage
    - File descriptors and open connections
2. Network performance
    - Throughput and packet loss
3. Storage metrics:
    - IOPS and latency
    - Read/write throughput
4. Virtualisation layer metrics (if applicable)
    - Hypervisor metrics like CPU ready/CPU contention metrics, memory ballooning, and so on

When running Nomad Enterprise on bare metal, host the metric collection using Nomad Enterprise itself (using an agent running as a `system` job) or using the underlying host OS's service management (such as `systemd`). When running Nomad Enterprise on virtualized infrastructure, collect the virtualization layer metrics using the virtualization provider's APIs.

9.5 Who watches the watchman

If you are self-hosting the observability stack on top of Nomad Enterprise, it is crucial to monitor the monitoring system itself to avoid a Nomad Enterprise cluster outage taking out your ability to notice

it and debug it. Do this by having a separate Nomad Enterprise cluster running the monitoring stack, or by using a SaaS monitoring system for high-level health checks. This is important for the alerting system - if it goes down, one cannot know if anything else also goes down. "Dead man switches" (raising an alert if it does not receive a heartbeat signal from the monitoring system) are a good way to ensure that you are aware of the monitoring system being down.

9.6 Benchmarking Nomad Enterprise

Benchmarking Nomad Enterprise is a crucial part of setting up observability, and evaluating your setup. It allows you to understand the performance limits of your Nomad Enterprise cluster and identify bottlenecks before they impact production workloads.

HashiCorp provides a Nomad Benchmarking project to help you get started with Nomad cluster performance testing. Refer to this blog post from the Nomad team on the project and their results.

9.7 Reference/useful links

Documentation: Nomad metrics reference Documentation: Monitoring Nomad Blog: Logging on Nomad with Vector Blog: Logging on Nomad and log aggregation with Loki OpenTelemetry collector running in Nomad example Running the OpenTelemetry demo app on HashiCorp Nomad Datadog integration for Nomad Nomad integration for Grafana Cloud Filebeat autodiscover Prometheus configuration reference

10 Backups

Nomad Enterprise's state consists of the following components:

- Jobs
- Deployments
- Evaluations
- Allocations
- Nodes
- Variables
- Key Material to encrypt secrets such as Variables and to sign JWTs

Nomad Enterprise backs this information up when it generates a snapshot.

> Note
>
> The only exception is the keystore, if using the local key store, back it up separately.

You can trigger snapshot creation manually or automatically with the snapshot agent.

To support your company's RPO policy we recommend configuring Nomad snapshot agent to take periodic backups of the Nomad cluster state. Configure the snapshot agent to take backups at a specified interval and store them in S3 or other compatible remote storage, preferably with configured lifecycle policy. Configure the number of backups the snapshot agent retains.

If using the snapshot agent for backups, Nomad Enterprise restores the cluster back to the time when the agent took the snapshot. To pull individual items from a backup, restore the snapshot to an isolated cluster and use the API to retrieve individual items. You can only restore snapshots to the same Nomad Enterprise version which took the backup.

Use the `nomad operator snapshot restore` command to restore a snapshot.

11 Overview

Autopilot is a set of features to allow for automatic operator-friendly management of Nomad servers. It includes automated upgrades, monitoring the state of the Raft cluster, and stable server introduction. This allows the Nomad upgrade process to be simplified.

See the Autopilot tutorial page for a basic tutorial and see below for more detail.

11.1 Prerequisites

1. Only if you are running Nomad 1.3 or older, set `raft_protocol` to **3** on all server nodes because the default raft versions were less than 2. This may also apply if you have upgraded from an older version, but never upgraded `raft_protocol` to 3. If you are running an older Nomad version with the parameter set to 2 or lower, then work to upgrade Nomad Enterprise. Refer to the Raft Protocol Version Compatibility documentation for more details.
2. Add `autopilot{}` block and its parameters to the server node configuration before bootstrapping the cluster. Use the `nomad operator autopilot set-config` command as an

alternative, or use the `/v1/operator/autopilot/configuration` API endpoint. Refer to the Autopilot documentation for more information on how to enable Autopilot.

We recommend you to have a build pipeline with Packer to create golden images, deploy the machines with Terraform, and a configuration management tool such as Ansible to perform any OS level operations after the VM provisioning.

Use the pipeline to deploy Nomad VMs with the new Nomad version or to do any changes to the OS or Nomad configuration. See the *Using Terraform to Configure Nomad* section for more details and recommendations on the deployment process.

12 Example pipeline

Below is an example pipeline of what the upgrade process may look like with Autopilot, Packer, and HCP Terraform.

- Packer builds a new Nomad Enterprise machine image and stores it at an image registry. This example shows AWS EC2 AMI, but the same principles apply for vSphere Content Library, Azure Compute Gallery.
- HCP Terraform uses a new workspace to deploy a new set of Nomad server VMs.

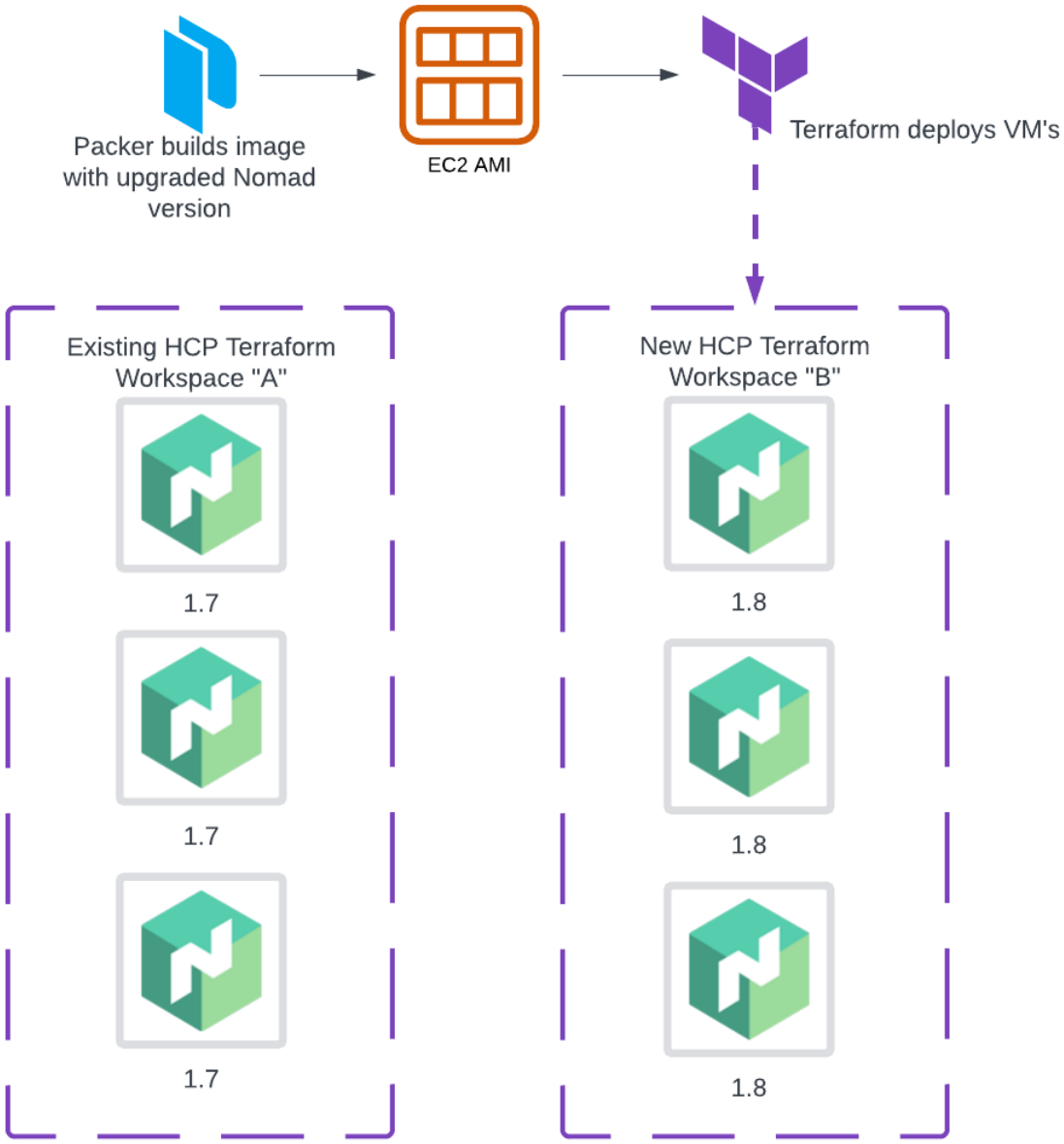


Figure 1: Terraform uses a new workspace to deploy a new set of Nomad servers.

- The new 1.8 nodes are joining the 1.7 cluster.

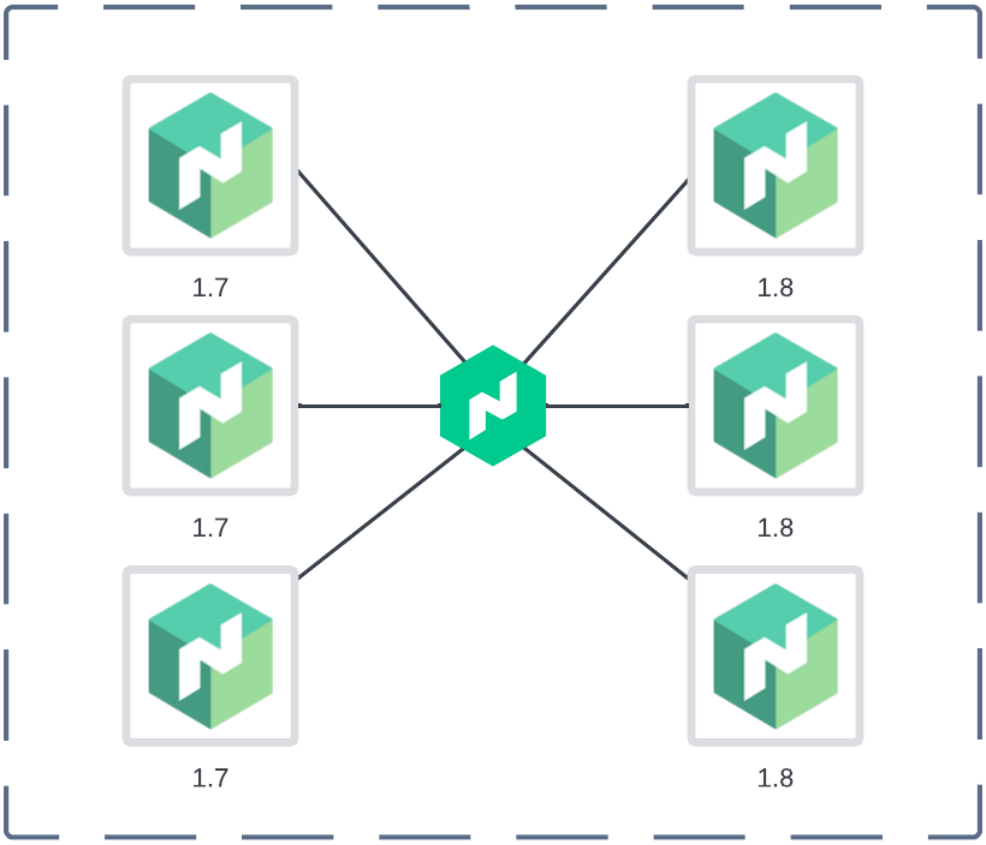


Figure 2: The new nodes are joining the cluster.

- Autopilot then automatically demotes the old server nodes to non-voting members so they are no longer participating in the quorum.

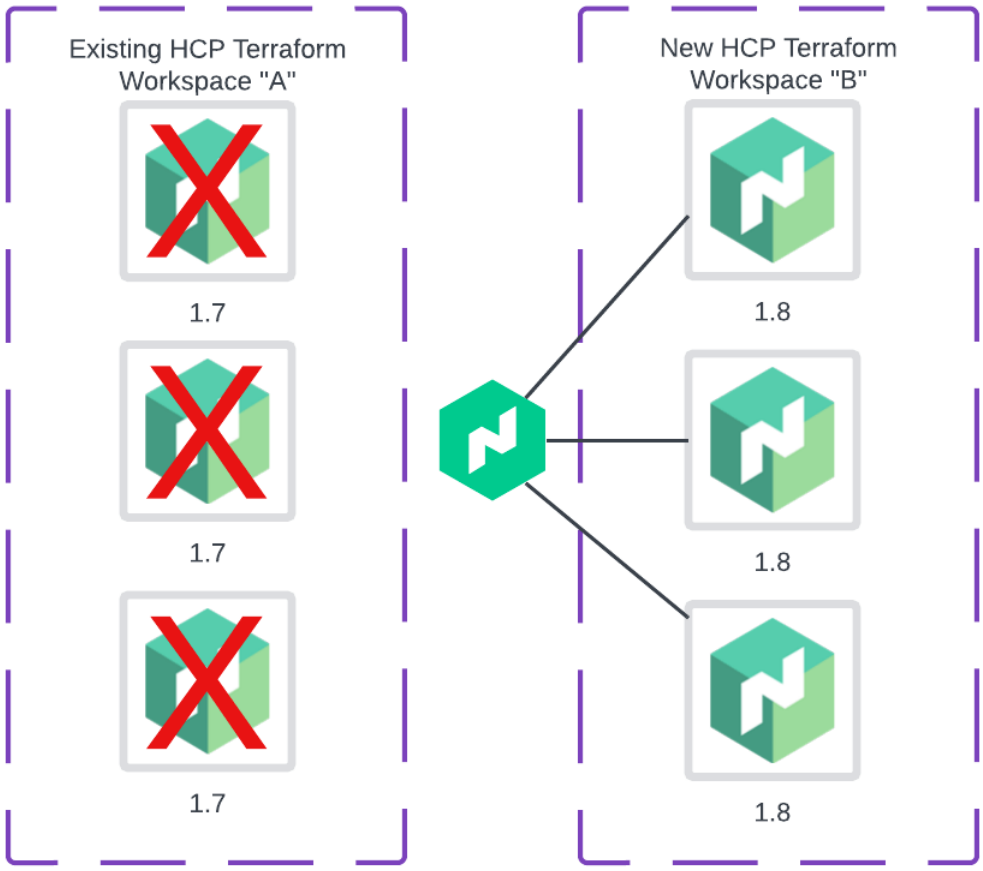


Figure 3: Autopilot automatically demotes the old server nodes.

Run `nomad operator raft list-peers` to confirm the Nomad nodes from *Workspace A* does not have a leader and are not set to voting members. Once you confirmed the Nomad instances deployed from *Workspace A* are non-voting, run `terraform destroy` to remove the old instances.

12.1 Autopilot recommended configuration options

12.1.1 Cleanup dead server

The `cleanup_dead_servers` parameter ensures that dead servers are automatically removed from the cluster. This is crucial for maintaining a healthy cluster state.

```
autopilot {
    cleanup_dead_servers = true
}
```

> Tip
>
> Always set this to `true` to prevent dead servers from causing issues in the cluster.

12.1.2 Last contact threshold

The `last_contact_threshold` parameter defines the maximum allowed time since the last contact with a server before Nomad Enterprise marks it unhealthy.

```
autopilot {
    last_contact_threshold = "200ms"
}
```

> Tip
>
> Set this to a low value (for example `200ms`) to detect and handle network partitions or server failures.

12.1.3 Server stabilization time

The `server_stabilization_time` parameter defines the time a server must be stable before Nomad Enterprise marks it as healthy.

```
autopilot {
     server_stabilization_time = "10s"
}
```

> Tip
>
> Set this to a reasonable value (for example `10s`).

12.1.4 Max trailing logs

The `max_trailing_logs` parameter specifies the maximum number of log entries a server can trail behind the leader before it is considered unhealthy.

```
autopilot {
    max_trailing_logs = 250
}
```

> Tip
>
> Start with 250 and adjust this value based on your cluster's workload and log generation rate. A higher value (1000) may be necessary for high-throughput environments. Monitor the cluster's performance and adjust the value based on your observations. If servers are frequently marked as unhealthy due to exceeding the max trailing logs, consider increasing the value.

12.1.5 Server stabilization time

The `server_stabilization_time` parameter specifies the minimum duration a server must be stable before Nomad Enterprise adds it to the cluster. The appropriate value depends on your server hardware and startup time.

```
autopilot {
    server_stabilization_time = "10s"
}
```

> Tip
>
> Use a lower value such as `10s` for servers with fast startup times and stable hardware. For servers with slower startup times or if you want to provide more time for servers to stabilize before joining the cluster, use a higher value such as `30s`. Observe the cluster's behavior during server additions and adjust if necessary.

12.1.6 Enable redundancy zones

The `enable_redundancy_zones` parameter allows you to enable redundancy zones, which improves the fault tolerance of your Nomad cluster by distributing voting servers across distinct failure domains. We recommend naming the `redundancy_zone` in your server configuration the same as the underlying availability zone.

```
autopilot {
    enable_redundancy_zones = true
}
```

> Tip
>
> Enable this feature to improve the resilience of your cluster by distributing servers across different failure domains. When enabling redundancy zones, ensure that you have sufficient servers distributed across multiple zones to maintain quorum and avoid data loss.

12.1.7 Disable upgrade migration

Set the `disable_upgrade_migration` parameter to false to allow automatic upgrades.

```
autopilot {
    disable_upgrade_migration = false
}
```

12.1.8 Disable upgrade migration and enable custom upgrades

`disable_upgrade_migration` and `enable_custom_upgrades`

> Tip
>
> **`false`** for both, unless you have specific upgrade requirements.

By default, Nomad Enterprise enables Autopilot's upgrade migration strategy, and disables custom upgrades. This allows Autopilot to automatically manage the upgrade process. If you have specific upgrade requirements or want to manually control the upgrade process due to just configuration changes without a newer version, you can set `disable_upgrade_migration` to **`true`** and `enable_custom_upgrades` to **`true`**. This allows you to implement your own upgrade logic.

For most cases, we recommend to leverage Autopilot's default upgrade migration strategy for a seamless and automated upgrade experience. If you choose to enable custom upgrades, ensure that you have `upgrade_version` set to your specific versioning semantics to allow you to increment the version tag on future images.

12.2 Summary

> Note
>
> Optimal settings may vary based on your specific environment, workload, and requirements. It is crucial to monitor your Nomad Enterprise cluster's performance, stability, and behavior and make adjustments as needed. Conduct thorough testing and gradually roll changes out to Autopilot parameters in a staging environment before applying them to your production cluster. This allows you to validate the settings and ensure a smooth operation.

12.3 Resource quotas

When many teams or users are sharing a Nomad cluster, there is the concern that a single user could use more than their fair share of resources. Resource quotas provide a mechanism for cluster administrators to restrict the resources within a namespace.

Once you attach a quota specification to a namespace, the Nomad Enterprise cluster counts all resource usage by job, in that namespace, toward the quota limits. If the resource exhaustion is experienced, allocations within the namespace queue until resources become available - by other jobs finishing or expanding quota.

We recommend that you enable resource quotas for shared environments where multiple teams or applications are running on the same Nomad Enterprise cluster. Below is a list of recommendations, however if you need a tutorial for how to implement, visit the Resource Quotas tutorial page.

12.4 Quota specification

Quota specifications are first class objects in Nomad Enterprise. A quota specification has a unique name, an optional human readable description, and a set of quota limits. The quota limits define the allowed resource usage within a region.

Quota objects are shareable among namespaces. This allows an operator to define higher level quota specifications. For example, a “Team-A” quota, and multiple namespaces can apply the “Team-A” quota specification.

Use the quota block in the Nomad job specification to define resource quotas. An example is below.

```
quota "team-a-quota" {
  limit {
    region = "global"
    region_limit {
      cpu = 2000
      memory = 4096
    }
  }
}
```

Note

It is crucial to properly design your namespace structure and workload placement within those namespaces, considering resource requirements and cluster capacity.

12.5 Applying quotas to namespaces

Once you define and apply the quotas, add them to namespaces. Below is an example of how to apply a quota to a namespace.

- Add to your specification file the quota parameter:

```
name        = "team-a-namespace"
description = "Namespace for Team A."
quota       = "team-a-quota"
```

- Run `nomad namespace apply ./anamespace.hcl`

12.5.1 Version control

We recommend keeping your namespace and quota specifications within version control for auditability and troubleshooting, and incorporate into a build pipeline that manages the deployment

of quotas such as Terraform. The Nomad provider provides a useful way to manage quotas and namespaces within your pipeline.

12.6 Monitoring

Monitor resource usage to ensure teams are within their quotas. There are several metric endpoints you can use to provide monitoring and alerts relating to quotas. Set the following block to **`false`** in your client configuration.

```
telemetry {
    disable_quota_utilization_metrics = false
}
```

Use the `nomad.nomad.blocked_evals.total_quota_limit` metric endpoint to alert you when jobs block due to reached quotas. Use `nomad.quota.utilization.cpu`, `nomad.quota.utilization.cores`, and `nomad.quota.utilization.memory_mb` for resource consumption quota management.

Ensure to set your filters based on quota name, namespace, or region to provide an accurate limit report.

12.7 ACLs

By implementing strict access control measures, you can prevent users from bypassing or modifying resource quotas without proper authorization. Accomplish this with the `quota{}` block within an ACL specification.

See the ACL section for more details.

12.8 Federated clusters

Nomad Enterprise replicates quota specifications in a federated cluster from the authoritative Nomad Enterprise region. This allows operators to interact with a single cluster but create quota specifications that apply to all Nomad Enterprise clusters. For example, you can create a single quota specification with multiple regions defined with their own limits.

```
name        = "federated-example"
description = "A single quota spec affecting multiple regions"

limit {
    region = "europe"
    region_limit {
        cpu = 20000
        memory = 10000
    }
}

limit {
    region = "asia"
    region_limit {
        cpu = 10000
        memory = 5000
    }
}
```

Applying this quota specification will apply it to all federated clusters.

12.8.1 Communication

Ensure that all teams are aware of their resource quotas and the importance of adhering to them. Regular communication or providing monitoring dashboards or alerting can help in avoiding any unexpected resource exhaustion and reduce the burden of the Nomad platform operators.

12.9 Using Sentinel policy as code for Nomad

Sentinel is HashiCorp's policy-as-code framework and language, and is part of Nomad Enterprise. It allows you to enforce fine-grained policies using business, security or compliance logic across your Nomad clusters.

By leveraging Sentinel, you can ensure compliance, security, and operational best practices are consistently applied to all workloads.

12.10 When to Use Sentinel and when to use access control lists

While both Sentinel and access control lists (ACLs) contribute to having a complete security and governance posture, they serve different purposes.

- **Use Sentinel** for complex, dynamic policies that require logic-based decision making, for example, enforcing resource quotas based on namespace or job name, allowlisting approved image registries, enforcing sensible defaults, or restricting when deployments can happen.
- **Use ACLs** for basic access control, such as determining which users or teams can submit jobs, or exec into them to specific namespaces.

Sentinel and ACLs are complementary. Use them in tandem for a comprehensive security and compliance strategy. ACLs provide the foundational access control, while Sentinel adds an additional layer of fine-grained policy enforcement.

12.11 Best practices for authoring Sentinel policies

When crafting Sentinel policies for Nomad Enterprise consider these recommended practices.

1. **Modularity**: Write focused policies that address specific concerns. This approach enhances readability and maintainability.
2. **Performance**: Be aware of the performance impact of Sentinel policies - all of them run, sequentially, on every job submission. Keep policies efficient, avoid unnecessary complexity and duplication of logic, and having too many small policies. As an example, condense two separate policies checking CPU and memory limits into a single one checking for both.
3. **Clear naming**: Use descriptive names for your policies and functions. For example, `enforce_resource_limits` is more informative than `check_resources`.
4. **Thorough testing**: Always test your policies against various scenarios to ensure they behave as expected. The Sentinel CLI and the Sentinel playground are invaluable tools for this purpose. To get mock data for realistic testing, you can use the Nomad Enterprise API to get the JSON definition an existing job, requiring conversion to snake case (example with jq) and put into a `job` object.
5. **Progressively Roll out policies**: When introducing new policies, consider rolling them out progressively to avoid sudden workflow disruptions. Start with `advisory` mode to give users the time to prepare and adjust, upgrade to `soft-mandatory`, which requires an extra ACL and flag/parameter to bypass, to enforce the policy but allow users to override, and then

switch to `hard-mandatory` to enforce the policy without exceptions. Note that as a Nomad operator, there is no way of tracking failed `advisory`-level policies. `Soft-mandatory` and `hard-mandatory` failures get reported in the audit logs.

6. **Treat Sentinel policies as code**: Version and track changes to your Sentinel policies in a git repository to facilitate collaboration, allowing rollbacks if needed. Implement a peer review process for policy changes to help catch potential issues and ensure policies align with organizational goals. Manage these using the Terraform provider for Nomad's nomad_sentinel_policy resource.

12.11.1 The new Sentinel UI in Nomad

Since Nomad Enterprise 1.8, there is now a dedicated Sentinel UI in the Administration tab, which enables you to manage and visualize your Sentinel policies directly within the Nomad UI. It has a policy overview, a policy writing UI with syntax highlighting and validation, and a template library to use.

12.12 Example Sentinel policies

Use the Sentinel UI in the Administration tab of the Nomad Enterprise UI for practical example policies. Also refer to the nomad-guides GitHub repository.

12.13 Control plane scaling

As your Nomad Enterprise deployment grows and evolves, you may need to scale your server nodes to maintain performance and reliability. This section outlines various strategies for scaling Nomad server nodes, each suited to different scenarios and requirements.

12.14 Metrics to monitor

Keeping an eye on the right metrics is crucial for making smart decisions about how to scale your Nomad cluster. While HashiCorp provides a comprehensive list of Key Metrics for overall cluster health and Server Metrics for server-specific insights, it is important to remember that every setup is unique.

Here are some general guidelines on which metrics typically point towards specific scaling methods. Keep in mind that these are just starting points – you'll need to fine-tune based on your particular needs and setup.

Horizontal scaling

- nomad.worker.invoke_scheduler.service
- nomad.worker.invoke_scheduler.batch
- nomad.worker.invoke_scheduler.system
- nomad.client.allocated.cpu
- nomad.client.unallocated.cpu
- nomad.client.allocated.memory
- nomad.client.unallocated.memory

Vertical scaling

- nomad.worker.invoke_scheduler.service
- nomad.worker.invoke_scheduler.batch
- nomad.worker.invoke_scheduler.system
- nomad.blocked_evals.total_blocked (if due to scheduler resource constraints)
- nomad.plan.wait_for_index (if due to leader resource constraints)
- nomad.plan.evaluate
- nomad.plan.queue_depth
- nomad.broker.total_unacked
- nomad.broker.total_pending
- nomad.plan.submit
- nomad.client.allocated.cpu
- nomad.client.unallocated.cpu
- nomad.client.allocated.memory
- nomad.client.unallocated.memory

Read-only replicas

- nomad.worker.invoke_scheduler.service
- nomad.worker.invoke_scheduler.batch

- nomad.worker.invoke_scheduler.system
- nomad.client.allocated.cpu
- nomad.client.unallocated.cpu
- nomad.client.allocated.memory
- nomad.client.unallocated.memory

Multi-region federated clusters

- `nomad.client.allocated.cpu` if consistently high across all nodes.
- `nomad.client.allocated.disk` if consistently high across all nodes.
- `nomad.client.allocated.iops` if consistently high across all nodes.
- `nomad.client.allocated.memory` if consistently high across all nodes.

12.15 Vertical scaling

When it comes to increasing the capacity of your Nomad Enterprise server nodes, vertical scaling involves adding CPU, memory, or disk IO to the underlying host.

For traditional datacenters, typically you would add resources to the VM. However, we recommend deploying new servers with the enhanced specifications. This method utilizes Autopilot and version tags.

1. Refer to the metrics listed above in vertical scaling section and when you notice an uptick in those metrics steadily over a period of time, then proceed with scaling the server nodes vertically.
2. Tag deployed server nodes with versions. This tag is a version number of your choosing that is not related to the Nomad Enterprise version. `nomad operator autopilot set-config -upgrade-version-tag=1.1.0`

   Instead, use the agent configuration.

   ```
   server {
     upgrade_version = "1.1.0"
   }
   ```

3. Launch new server nodes with increased CPU, memory, or disk resources; and increased version tag. For example, `upgrade_version=1.2`.

4. Add these new nodes into your existing Nomad Enterprise cluster.
5. Autopilot automatically adds the nodes and remove the old nodes from the cluster. You need to delete the VMs if you are using a CSP, ideally with a build pipeline.

It is important to note that vertical scaling is the most effective method for addressing control plane resource contention, as the scheduling engine must serialize through the broker and "plan applier" on the leader. However, you may encounter limitations such as reaching the maximum capacity of a single instance, prohibitive costs, or the need to meet high availability requirements. In such cases, it is advisable to consider horizontal scaling strategies, implementing read-only replicas, or exploring the option of federated clusters for your Nomad infrastructure.

12.16 Horizontal scaling

Horizontal scaling is a concept of adding nodes to the existing cluster allowing distribution of traffic across joined nodes.

12.16.1 Prerequisites

- Refer to the metrics listed in the preceding horizontal scaling section. When you notice an uptick in those metrics steadily over a period of time, then proceed with scaling the server nodes horizontally.
- An existing AMI or image template created and available. Use Packer to make this process scalable.
- Leveraging Terraform and the relevant providers can make the implementation much quicker. Configuring these components from scratch is out of scope of this document, however below are links get you started.

Should the Nomad Autoscaler be used?

It is not recommended to use the Autoscaler for scaling the server nodes from 3 to 5. There are several reasons for this:

- Once you scale up to 5 server nodes, it is often unnecessary and potentially destabilizing to scale back down.

- The Autoscaler lacks mechanisms to ensure the Raft leader is not inadvertently removed during a scale-down operation, which could disrupt cluster stability.
- Managing a fluctuating number of server nodes can introduce unnecessary complexity in maintaining quorum and overall cluster health.

Instead, it is generally recommended to only increase the number of server nodes using Terraform based on careful evaluation of your cluster's performance and needs. This approach allows for more controlled and deliberate scaling decisions, ensuring the continued stability and efficiency of your Nomad cluster.

12.17 Enhanced read-only replica nodes

Nomad Enterprise provides the ability to scale clustered Nomad servers to include voting servers and non-voting read replicas. Read replicas still receive non from the cluster replication, however, they do not take part in quorum election operations and are not promoted by Autopilot. Expanding your Nomad cluster in this way can scale read operations without impacting write latency.

12.17.1 Configure read-only replica nodes

Add the following configuration to your Nomad server configuration file of nodes you want to perform read operations:

```
server {
    enabled = true
    non_voting_server = true
}
```

Restart the Nomad server to apply the changes.

Always monitor your read-only nodes and scale as needed.

12.17.2 Autoscaler

Unlike Horizontal scaling, use the autoscaler to scale up and down read-only nodes due to the nodes not participating in leader voting.

> **Note**
>
> The autoscaler only applies to Nomad server & client nodes hosted on AWS, Azure, or GCP.

12.17.3 Prerequisites

- Familiarize yourself with the concepts and complete the tutorial.
- Refer to the preceding metrics in the read-only scaling section.
- An existing AMI or image template created and available. Ensure this template has the `non_voting_server = true` parameter listed in the server configuration. Use Packer to make this process repeatable.
- If using redundancy zones, set the `redundancy_zone` and your Terraform deployment pipeline supports this.
- Leveraging Terraform and the relevant providers can make the implementation much quicker. Configuring these components from scratch is out of scope of this document, however below are links to get you started.

Set up a Nomad Enterprise cluster on GCP.

- AWS Terraform provider resources
  - aws_ami
  - aws_autoscaling_group
- AWS tutorials
  - Manage AWS Auto Scaling Groups
  - Set up a Nomad cluster on AWS
- Azure Terraform provider resources
  - azurerm_image
  - azurerm_linux_virtual_machine_scale_set
- Azure Tutorials
  - Manage Azure Virtual Machine Scale Sets with Terraform
  - Set up a Nomad cluster on Azure
- GCP Terraform provider resources

    - google_compute_image
    - google_compute_autoscaler
- GCP Tutorials
    - Set up a Nomad cluster on GCP

12.17.4 Example read only Autoscaling scaling policy for server nodes

Remember, Autoscaling is an art. Every use case and environment is unique and utilizes a variety and combinations of APMs, strategies, and targets. These require fine tuning. Some metrics may not be sufficient on their own and require a secondary metric to truly determine whether to trigger the Autoscaler or not.

Always take a proactive approach when Autoscaling. Never wait for issues to present themselves before Autoscaling as that can possibly result in the Autoscaler to fail the scaling event or cause downtime to workloads.

The example below evaluates resource constraint on the server nodes. It triggers a scaling event on an AWS ASG target if all `nodesng` conditions are true.

- Memory usage > 80%
- CPU usage > 80%
- Golang routines spike 50% over 5 minute period.

Remember, this example provides as a starting point, sufficient for most read-only server node scaling use cases.

```
template {
        data = <<EOF
scaling "cluster_policy" {
  enabled = true
  min     = 0
  max     = 5

  policy {
    cooldown            = "2m"
    evaluation_interval = "1m"

    check "cpu_allocated_percentage" {
      source = "prometheus"
      query  = "sum(nomad_client_allocated_cpu{}*100/(nomad_client_unallocated_cpu{}+
          nomad_client_allocated_cpu{}))/count(nomad_client_allocated_cpu{})"

      strategy "threshold" {
        lower_bound = 80
      }
    }

    check "mem_allocated_percentage" {
      source = "prometheus"
      query  = "sum(nomad_client_allocated_memory{}*100/(
          nomad_client_unallocated_memory{}+nomad_client_allocated_memory{}))/count(
          nomad_client_allocated_memory{})"

      strategy "threshold" {
        lower_bound = 80
      }
    }

    check "goroutine_spike" { source = "prometheus" query = "100 *  max(
        nomad_runtime_num_goroutines{}) / max(nomad_runtime_num_goroutines{} offset 5
        m) - 1)"

        strategy "threshold" {
            upper_bound = 50
            delta = 1
        }
    }

    target "aws-asg" {
      dry-run             = "false"
      aws_asg_name        = "nomad-servers"
      node_drain_deadline = "5m"
    }
  }
}
EOF
      destination = "${NOMAD_TASK_DIR}/policies/hashistack.hcl"
      }
```

12.18 Federated clusters

Federated clusters allow multiple Nomad Enterprise clusters to work together, providing high availability and disaster recovery. By deploying Nomad Enterprise clusters across multiple regions, users can interact with any Nomad Enterprise server endpoint, regardless of its location. This provides a single pane of glass irrespective of which cluster they connect to.

> Warning
>
> When enabling ACLs, Nomad Enterprise depends on an authoritative region to act as a single source of truth for ACL policies and global ACL tokens. Nominate one of your regions as authoritative in this scenario.

Refer to this tutorial on how to setup federation.

Consider implementing federated clusters when:

- You have workloads in different regions or datacenters that you want to manage centrally.
- Your single Nomad Enterprise cluster is struggling to handle the scale of your operations across multiple locations.
- You need to maintain separate clusters for compliance or organizational reasons, but still want a unified view and management capability.
- You are looking to improve fault tolerance and disaster recovery by distributing your workloads across multiple clusters.

Migrate a job to another cluster

Once the primary cluster federates a new cluster, update the jobs need to deploy to the new cluster. Detailing a job migration is outside the scope of this document, as each organization has vastly different and nuanced requirements and processes. If there is an existing deployment pipeline, the process is quite straight forward.

The most difficult part is meeting underlying infrastructure and application dependencies, such as all networking and firewall rules. Existing ACL and Sentinel policies replicate automatically.

Within Nomad Enterprise, update or add the datacenter and region block to your job file.

12.18.1 Scaling strategy summary

The following table presents scaling strategies in the typical order of implementation as your Nomad cluster grows and evolves.

Strategy	Pros	Cons	Best for
Vertical Autoscaling	Simplest way to increase capacity	Cap to how large you can scale	You need capacity increases without the increased cost of adding nodes
Horizontal Autoscaling	Easy to implement with no downtime	Increased instance cost, not recommended to scale passed 5 nodes	If current server node count is 3, instance sizes are near maximum, and you are observing performance bottlenecks
Read-only replicas	Improved read scalability, Reduced write contention	Additional resource overhead	You've reached vertical and horizontal scaling limit, large scheduling events or other read operation bottlenecks
Federated clusters	Global scale, improved fault tolerance	Increased operational complexity	Multi-region deployments, large enterprise footprint

Scaling Nomad Enterprise server nodes is a crucial aspect of maintaining a healthy and efficient cluster as your workload demands increase. Remember that each scaling strategy has its own pros and cons, and the best approach depends on your specific use case, growth trajectory, and operational capabilities.

Tip

Before scaling in production, it is *crucial* to thoroughly test the changes in a development or test environment. Make this a requirement in your project plan. This allows you to assess the impact on your entire infrastructure, including networking, storage, and any other systems that interact with your Nomad cluster. Use this testing phase to identify and resolve any potential issues, and to fine-tune your approach.Ensure that you have a robust backup strategy in place before proceeding. Create and verify snapshots of your Nomad Enterprise state, as this is invaluable for reliable recovery if needed during the scaling process.Once you are confident in the results and have confirmed that your backup process is working correctly, update your disaster recovery and backup strategies to account for the new server specifications.Only then proceed with scaling your production environment, knowing you have a safety net in place.

13 Changelog

13.1 2026-08

- Restructured legacy Nomad HVD content into the HVD 2.0 Nomad Administration Guide as part of the Installation / Administration / User guide migration.

13.2 2026-01

- Update meta descriptions
- Updated credits and contributors
- Fix threshold descriptions

13.3 2025-10

- Review of Nomad Operating Guide

13.4 2025-07

- Updated documentation links

13.5 2025-05

- Removed document structure section
- Fixed code block formatting

13.6 2025-04

- Updated documentation links

13.7 2025-02

- Update introduction

14 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

- Adrian Todorov
- Amber Reddy
- Andrei Burd
- Mark Lewis
- Prakash Manglanathan
- Shaun Coss